When should reflection restrain the first impulse?

Measured results from the Nafs-Aql System 1 / System 2 laboratory on a 4-core CPU

Dirac R&D Team

18 October 2026

Abstract

Nafs-Aql pairs a fast small language model (System 1, Qwen3-1.7B) with a slower reasoning model (System 2, Qwen3-4B-Thinking-2507) behind a gate that decides when to deliberate, and treats exact verifiers, not model agreement, as the evidence of correctness. This note reports what the running system measured on a shared Intel i3-8100 (4 cores, AVX2, 32 GB). Every number below is generated from stored evaluation runs by `scripts/gen_publication.py`, and each table names its run identifiers. The headline results are indicative: release gates require 700 to 6 200 paired items, while one CPU produced tens. With an exact verifier visible, the gated system matched deliberate-only accuracy at 0.36 of its cost; with the verifier hidden, the uncalibrated gate routed correctly on only 50% of items, because cheap samples agreed on wrong answers. The vocabulary of the self (*nafs*, *'aql*) is used throughout as a design analogy, not as a religious claim; its text awaits scholar review.

1 System and laboratory

A request is drafted by System 1 (state A, *nafs*). The Aql Gate escalates when the expected error saved outweighs the price of the extra latency, $L(\hat{e}_1 - \hat{e}_2) > \lambda \Delta c$; an exact-verifier failure always escalates (state L, *lawwma*). System 2 then reasons within a thinking budget without seeing the draft (state M, *'aql*); an answer is accepted with an assurance level (T, *mumainna*) or the system abstains (X, *tawaqquf*). The laboratory compares *policies* for combining the two systems on identical tasks: fast-only, deliberate-only, the gate with the verifier visible, the gate routing on its own estimate ("blind"), a draft-conditioned ablation, budget sweeps and λ sweeps, plus human "Both" trials in chat. Paired differences carry bootstrap intervals, exact McNemar tests and Bonferroni correction.

2 Results

2.1 Policies on a mixed task set (run `7ffd4442`)

System	n	Acc.	95% CI	Esc.	Wall (s)
fast_only	6	67%	[22%, 96%]	0%	13
nafs_aql	6	100%	[54%, 100%]	33%	96
nafs_blind	6	67%	[22%, 96%]	17%	133
slow_conditioned	6	100%	[54%, 100%]	100%	319
slow_only	6	100%	[54%, 100%]	100%	264

Exact-verifier accuracy with Clopper–Pearson intervals (6 tasks per system: GSM8K, MBPP, JSON Schema). The gate with the verifier visible equals deliberate-only accuracy at a cost ratio of 0.36. The blind gate kept two wrong math drafts whose samples agreed with them (routing accuracy 50%): sample agreement is a flag, not evidence, and calibration on more labelled data is the remedy.

2.2 Verifier quality (runs `de66e962`, `f7991b65`, `50ff0d27`, `fd07e0a8`, `dc7168b0`)

Verifier	Correct	r	Mutants	φ	φ upper
code_tests	364	0.995	754	0.109	0.133
json_schema	24	1.000	53	0.000	0.067
judge	23	1.000	22	0.045	0.228
math_numeric	2000	1.000	2800	0.000	0.001

Correct answers give the true-accept rate r; seeded wrong answers (mutants) give the false-accept rate φ with a 95% upper bound. Math meets the $\varphi \leq 0.01$ target. The code φ is an upper bound inflated by behaviourally equivalent mutants: the survivors pass the much larger EvalPlus test suites as well. Measuring on EvalPlus exposed three verifier defects (a Python long-line encoding failure and two resource limits), all fixed. The rubric judge returned valid verdicts on all items once its output was grammar-constrained; its one false accept was a factual error about authorship.

2.3 Speed on the CPU

Skipping cheap System 1 samples when an exact verifier decides cut the median draft time from 13.4 s to 7.9 s. Speculative decoding with a 0.6B draft model, or with n-gram lookup, did not help (median 6.5 and 6.6 against 7.3 tokens/s): on four cores the draft competes with the main model for compute and memory bandwidth. The load-test protocol (run `6c462074`) gives:

Sessions	p50 (s)	p95 (s)	p99 (s)	req/s
1	3.8	7.2	7.2	0.25
2	2.7	4.9	7.2	0.68
4	3.6	9.0	12.0	0.76

NFR-007 asks that p95 at four sessions stay within twice the single-session value; the measured ratio is 1.25, so the requirement fails on this shared host and concurrency is documented as a larger-machine feature.

2.4 Where the time goes (Phase 7)

Every reply records its stages, and model stages carry llama-server's own prefill, decode and KV-cache figures (514 replies):

Path (replies)	Stage	Share	p50 (s)	p95 (s)
fast (475)	System 1 draft	98%	13.6	56.1
fast (475)	verify draft	2%	0.2	0.9
deliberate (39)	System 2 deliberation	95%	120.6	424.1
deliberate (39)	judge	3%	40.0	44.3
deliberate (39)	System 1 draft	2%	9.2	59.2
deliberate (39)	probe	1%	1.5	6.4

Model stage	Calls	Prefill tok/s	Decode tok/s	Cache reuse
System 1 draft	485	53	7.0	60%
cheap samples	32	13	4.2	80%
System 2 deliberation	45	16	3.4	95%

The fast path is almost entirely System 1 decoding; the prompt cache already serves most prompt tokens. Early escalation stops an open-ended draft whose running mean log-prob is already below the fixed rule's threshold: on 40 open-ended prompts, the configured rule ($N = 32$, margin 0.04) agreed with the full-draft decision on 100% of prompts, caught 3 of 10 escalations early and saved 116 draft tokens per early escalation.

N	Margin	Agreement	Caught	False	Saved tok/esc.
32	0.00	80%	10/10	8	82
32	0.02	95%	5/10	2	92
32	0.04	100%	3/10	0	116
32	0.15	100%	0/10	0	0
48	0.00	90%	9/10	4	77
48	0.02	98%	4/10	1	117
48	0.04	100%	2/10	0	154
48	0.15	100%	0/10	0	0

2.5 Regression baseline and model choice

Fast-path baseline on the fixed 500-item regression suite (run `880cdb33`): 500 items scored (done), fast-path accuracy 69% [65%, 73%], mean wall 24.5 s. Profile C (Qwen3.5) decision: Keep Profile A (Qwen3-1.7B) as System 1 default: accuracy 83% vs 67% (delta +17%, 95% CI [+7%, +30%]), mean latency 88.9 s vs 24.0 s on 30 tasks (shared machine). System 2 (Qwen3.5-4B) not evaluated at this scale; GPU scale-out (P5-T03) deferred by the user.

System 1	All	Math	Code	JSON	Wall (s)	tok/s
Qwen3-1.7B (Profile A)	67%	56%	75%	83%	24.0	4.3
Qwen3.5-2B (Profile C)	83%	81%	75%	100%	88.9	1.9

2.6 Operations

26/26 security tests pass; 0 open high-severity findings; one high-severity issue (sandboxed code could signal other processes) was found by the suite and fixed. Deployment recipes: 11 passed, 0 failed, 1 skipped (tool or host not available).

3 Roadmap gates

Gate	Status	Evidence
G0	pending	H7: pass; P0-T04: pending
G1	fail	MVP: fail; NFR-002/004: fail
G2	pending	ECE: pending; H1: pending; ER-002: pending; ER-004: pending
G3	pending	TR-007: pending; OR-005: pending
G4	fail	P4: pass; ER-007: pass; NFR-007: fail
G5	pass	SR-006/SR-002: pass; OR-003: pass; MR-005: pass
G6	pending	ER-002: pending; ER-008: pass; SR-005: pending

4 Limitations

All measurements come from one shared 4-core machine whose other workloads raised the load average to 6–15, so absolute speeds are lower bounds and small effects are within noise. Sample sizes are far

below the release gates; intervals, not point estimates, should be read. GPU training (Phase 3) and GPU scale-out were deferred. The analogy text has not yet been reviewed by a scholar of the tradition.

5 Reproducibility

Runs, decisions and traces are stored in `data/runtime/nafs.sqlite`; models are pinned by SHA-256 in `config/models.manifest.json`; regenerate these tables with `python3 -I scripts/gen_publication.py` and the PDF with `tools/tectonic docs/publication/nafs_aql_lab_results.tex`.